

Week 1 – Lesson Content

Hinweis zu fehlenden Lektionen: Die Notebook-basierten Lektionen (*Python Review Notebook*, *ML & Neural Networks Notebooks Day 2+3*, *Generative AI Notebook*, *Speech Recognition Notebooks*) sind im Kurstext nicht als Fließtext enthalten und daher hier nicht aufgeführt.

Day 1 | AI Literacy

AI Literacy Overview

AI literacy extends far beyond text generation, encompassing the broader landscape of artificial intelligence technologies and their applications. Understanding the rise of generative AI and its place within the larger AI ecosystem is essential for developing a comprehensive view of the field.

Types of AI

Artificial intelligence can be categorized in multiple ways, each highlighting different aspects of capability, scope, and application. These classifications help us understand where current technologies stand and what future developments might look like.

Moravec's Paradox

Moravec's Paradox reveals an interesting inverse relationship: tasks that are easy for humans are often difficult for AI, while tasks that are hard for humans can be relatively easy for AI. For example, chess mastery and complex coding are easier for AI, while folding laundry, sewing, and walking on uneven terrain remain challenging. Massive data analysis is where AI excels, demonstrating the counterintuitive nature of AI capabilities.

AI-Complete Problems

An AI-complete problem is one that requires full human-level intelligence to solve, paralleling the concept of NP-complete problems in computer science. These are problems that cannot be solved with a specific algorithm alone and may require the development of Artificial General Intelligence (AGI) to address fully.

AI Classifications

Narrow AI Narrow AI systems are designed for one specific task, such as chess engines, image classifiers, or cancer prediction models. They excel at their designated function but cannot transfer knowledge to other domains.

Weak AI Weak AI systems can handle multiple tasks but within a limited domain, such as virtual assistants like Siri or Alexa. They demonstrate versatility within their constrained scope but lack general intelligence.

Artificial General Intelligence (AGI) AGI represents human-level reasoning with the ability to transfer learning across domains. Estimates for achieving AGI range from 2030 to 2050, though this remains uncertain.

Artificial Super Intelligence (ASI) ASI would surpass human capability, with the ability to self-improve and potentially raise profound philosophical implications about the future of intelligence and humanity.

Strong AI The term "Strong AI" is often used interchangeably with AGI, referring to systems that possess general intelligence comparable to human cognition.

Large Language Models (LLMs)

Large Language Models are challenging to categorize neatly within traditional AI classifications. They are more capable than older narrow AI models but do not represent true AGI. Sometimes referred to as "broad AI," LLMs demonstrate impressive capabilities across multiple domains while still falling short of general intelligence.

Agentic AI

Agentic AI represents a shift from simple question-answer systems to AI that can perform jobs autonomously. Unlike traditional AI that responds to queries, agentic AI follows a process of question → plan → execute → plan, enabling it to work independently toward goals. Examples include AI coding assistants and autonomous research agents that can plan and execute complex tasks.

AI Consulting Framework

Before starting any AI project, it's crucial to break down the problem systematically. The consulting framework involves asking key questions: Has this problem been solved before? What type of AI does it require? What are the consequences if it fails? Will it be able to scale? Is this the organization's first interaction with AI? These considerations help ensure appropriate AI solutions and manage expectations.

Day 1 | Python Environment Setup

Introduction

Setting up your development environment is the foundation of everything you'll do in this bootcamp. Having the right tools configured correctly from the start will save you countless hours and prevent frustrating errors down the road.

Real-world context: Every developer needs a properly configured workspace. The setup is similar to preparing your kitchen before cooking. You need the right ingredients (tools), organized workspace (VSCode), and proper storage (GitHub) before you can create something amazing.

VSCode Setup

Visual Studio Code (VSCode) is a powerful, free code editor that provides excellent support for Python, Jupyter notebooks, and Git integration.

GitHub Account Setup

GitHub is like Google Drive, but for code – with the ability to go back in time to see every version. It's an essential tool for collaboration, version control, and showcasing your work.

Understanding Git Basics

- **Stage (git add):** Mark changes to be saved. Think of it as selecting which files you want to keep.
- **Commit (git commit):** Save your staged changes with a message describing what you did. This creates a way to find them afterwards.
- **Pull (git pull):** Get the latest version of files from GitHub to your computer.
- **Push (git push):** Send your committed changes from your computer to GitHub.

Conda Environment Management

For each project or module, it's necessary to isolate it in its own environment. Environments are separate pockets on your computer that keep project dependencies separate so they don't interfere with each other. This ensures that everyone in the company working on the same project shares the same configuration.

Why isolation matters:

- Different projects may require different versions of the same package
- Installing everything globally can cause conflicts
- Environments ensure your projects stay intact and reproducible

Jupyter Notebooks in VSCode

Jupyter notebooks are interactive documents that combine code, output, and markdown. They are more friendly for experimentation and make taking notes easier.

Day 1 | Python Review

Variables

Variables store data values. Python automatically determines the type based on the value you assign.

```
name = "Python"
age = 30
price = 19.99
is_active = True
result = age + 10    # 40
message = f"Hello, {name}" # "Hello, Python"
```

Lists

Lists are ordered collections of items. They're mutable (you can change them) and can contain different data types.

```
fruits = ["apple", "banana", "orange"]
numbers = [1, 2, 3, 4, 5]
first_fruit = fruits[0] # "apple"
last_fruit = fruits[-1] # "orange"
fruits.append("grape")
fruits.pop()
fruits.insert(1, "mango")
```

Dictionaries

Dictionaries store key-value pairs. They're perfect for representing structured data like API responses or configuration settings.

```
student = {
    "name": "Alice",
    "age": 25,
    "course": "AI Bootcamp"
}
name = student["name"]    # "Alice"
age = student.get("age")  # 25 (safer method)
student["email"] = "alice@example.com"
del student["age"]
keys = student.keys()
values = student.values()
```

Sets

Sets are unordered collections of unique elements. They're useful for removing duplicates and performing set operations.

```
numbers = {1, 2, 3, 4, 5}
colors = {"red", "blue", "green"}
numbers.add(6)
numbers.remove(3)
union = numbers | {7, 8}
intersection = numbers & {4, 5, 6}
```

If/Else

Conditional statements let your code make decisions based on conditions.

```
age = 20
if age >= 18:
    status = "adult"
else:
    status = "minor"
```

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
```

For Loops

For loops iterate over sequences like lists, strings, or ranges.

```
fruits = ["apple", "banana", "orange"]
for fruit in fruits:
    print(fruit)
```

```
for i in range(5):
    print(i)
```

```
for index, fruit in enumerate(fruits):
    print(f'{index}: {fruit}')
```

While Loops

While loops continue executing as long as a condition is true. Use them when you don't know how many iterations you need.

```
count = 0
while count < 5:
    print(count)
    count += 1

while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == "quit":
        break
```

Functions

Functions are reusable blocks of code that perform specific tasks.

```
def greet(name):
    return f"Hello, {name}!"

message = greet("Alice") # "Hello, Alice!"

def calculate_total(price, tax_rate=0.1):
    return price * (1 + tax_rate)

total = calculate_total(100) # 110.0
total_with_custom_tax = calculate_total(100, 0.15) # 115.0
```

Classes

Classes allow you to organize code into reusable components called objects. They help you model real-world entities, structure complex projects, and create tools that can be reused across different parts of your code.

When it's used: building data models (like representing a user or a product), creating reusable tools (like a data processor or API client), and structuring complex projects where you need to group related data and functions together.

```
class Student:
    def __init__(self, name, age, course):
        self.name = name
        self.age = age
        self.course = course
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)

    def get_average(self):
```

```

        if len(self.grades) == 0:
            return 0
        return sum(self.grades) / len(self.grades)

    def introduce(self):
        return f"Hi, I'm {self.name}, {self.age} years old, studying {self.course}"

student1 = Student("Alice", 25, "AI Bootcamp")
student1.add_grade(85)
student1.add_grade(90)
student1.add_grade(88)

print(student1.introduce())    # "Hi, I'm Alice, 25 years old, studying AI Bootcamp"
print(f"Average: {student1.get_average()}") # 87.67

```

Key concepts:

- `__init__`: The constructor method that runs when you create a new object
- `self`: A reference to the instance of the class (the object itself)
- **Methods**: Functions defined inside a class that operate on the object's data
- **Attributes**: Variables that belong to the object (like `self.name`, `self.age`)

Import Libraries

Python's power comes from its extensive library ecosystem. Libraries like pandas and json are essential for data science and API work.

pandas – Data analysis, cleaning datasets, reading/writing CSV files, manipulating structured data.

```
import pandas as pd
```

```

data = {
    "name": ["Alice", "Bob", "Charlie"],
    "age": [25, 30, 35],
    "score": [85, 90, 88]
}
df = pd.DataFrame(data)

print(df.head())
print(df["age"].mean())
print(df[df["score"] > 85])

df_from_file = pd.read_csv("data.csv")
df.to_csv("output.csv", index=False)

```

json – Parsing API responses, reading/writing JSON configuration files, data serialization.

```
import json
```

```
json_string = '{"name": "Alice", "age": 25, "course": "AI Bootcamp"}'  
data = json.loads(json_string)  
print(data["name"]) # "Alice"
```

```
student_dict = {"name": "Bob", "age": 30, "grades": [85, 90, 88]}  
json_output = json.dumps(student_dict)
```

```
with open("data.json", "r") as file:  
    data_from_file = json.load(file)
```

```
with open("output.json", "w") as file:  
    json.dump(student_dict, file, indent=2)
```

Common import patterns:

```
import pandas as pd          # Standard import  
from json import loads, dumps # Import specific functions  
from math import *           # Import everything (not recommended for large libraries)
```

Day 2 + 3 | Neural Networks Fundamentals

Non-Generative AI in Practice

While generative AI may be the more exciting technology, non-generative AI plays a crucial role in practical applications. Non-generative AI is easier to control, maintain, and predict, making it essential for production systems, and it can also be integrated with generative AI to enhance capabilities.

Examples of Non-Generative AI Applications: credit card fraud detection, customer churn prediction, disease diagnosis from medical records, loan approval prediction, sales and energy forecasting, weather prediction, customer segmentation, image compression, social network community detection, market basket analysis, facial recognition, medical image diagnosis, quality inspection in manufacturing, satellite image analysis, and energy load forecasting.

Scikit Learn: Classic AI in Practice

Scikit Learn is the primary tool for implementing classic AI solutions, focusing on two main components: data processing and model building. The framework provides a comprehensive set of tools for both supervised and unsupervised learning tasks.

Model Building: Choosing the Right Model

All models learn a function $f(x)$, but they do so in different ways depending on the algorithm.

Key Decision Factors The two most important questions when choosing a model are: "Do you have a label?" and "Are you predicting a category or a quantity?" These questions determine whether you need supervised or unsupervised learning, and whether you're doing classification, regression, clustering, or dimensionality reduction.

Supervised Learning Supervised learning requires labeled data and is used when you have known outcomes. If you're predicting a category, you use classification; if you're predicting a quantity, you use regression.

Unsupervised Learning Unsupervised learning works with unlabeled data to discover patterns. If you're looking for categories in your data, you use clustering; otherwise, you use dimensionality reduction.

Model Categories

Distance/Math-Based Models Distance-based models make predictions based on similarity between data points, assuming that points close to each other are similar. These models include SVM, K-Nearest Neighbors, Neural Networks (MLP), and PLS, and they require careful preprocessing with everything on the same scale.

Linear Models Linear models assume a linear relationship between data and prediction values ($f(x) = ax + b$). Examples include Linear Regression, Logistic Regression, Regularized Models, Poisson Regression, Multinomial Regression, GAMs, and Discriminant Analysis, which need preprocessing for multicollinearity and scale issues.

Tree-Based Models Tree-based models split data using decision "cuts" and can handle almost any type of data. Examples include Random Forest, Gradient Boosting, Decision Trees, BART, Bagged Trees, C5.0 Rules, and Cubist Rules. They generally need minimal preprocessing as they're robust to scale and can handle raw data well.

Specialty Models Specialty models don't fit neatly into other categories and include MARS, Bagged MARS, Naive Bayes, and RuleFit. Each has specific preprocessing requirements depending on the model type.

Model Evaluation Metrics

Classification Metrics

- **Accuracy** – works for balanced datasets
- **Precision** – when false positives are expensive
- **Recall** – when false negatives are expensive
- **F1 Score** – for balanced performance on imbalanced data
- **AUC-ROC** – for comparing models or choosing thresholds
- **Macro F1** – for multi-class problems where classes are equally important
- **Weighted F1** – for multi-class problems accounting for class imbalance

Regression Metrics

- **RMSE** – default choice, penalizes large errors
- **MAE** – robust to outliers
- **R²** – helps explain variance to stakeholders
- **MAPE** – useful for comparing across different scales

Clustering Metrics

- **Silhouette Score** – general quality check (aim for > 0.5)
- **Elbow Method** – helps choose the number of clusters
- **ARI** – used when you have true labels to compare against

Red Flags

- 99% accuracy on imbalanced data → useless model
- $R^2 = 1.0$ → likely overfitting
- High precision with low recall → missing many cases
- RMSE much larger than MAE → significant outliers

Always use multiple metrics to get a complete picture of model performance.

Deep Learning: Principles and Applications

Why Deep Learning? Deep learning is essential when your data isn't in a table format. It excels at processing images, text, audio, and other unstructured data types that traditional machine learning models struggle with.

Neural Network Fundamentals Neural networks are inspired by biological neurons and consist of interconnected nodes that process information. The network learns to approximate complex functions $f(x)$ through layers of neurons, with each neuron performing weighted computations and applying activation functions.

TensorFlow vs Hugging Face

- **TensorFlow** – for researchers and engineers who want to build models from scratch
- **Hugging Face** – for those who want to use existing models or adapt pre-trained models to specific use cases

Deep Learning Vocabulary

- **Epoch** – one complete pass through all training data (like reading a textbook once)
- **Batch** – small groups of examples processed together (like one chapter at a time)
- **Learning Rate** – the size of steps taken during optimization; too small = slow learning, too large = overcorrecting

Day 4 | Generative AI

Introduction to Large Language Models

Large Language Models (LLMs) are massive neural networks with billions or trillions of parameters, representing a significant evolution from classic AI systems with thousands of parameters to modern GPT models with 1.7 trillion parameters.

The Scale of AI Evolution

The progression from classic AI (10k parameters) to modern GPT models (1.7 trillion parameters) demonstrates exponential growth in model complexity, with each parameter representing approximately 1 cm in a physical scale comparison.

Encoder-Decoder Architecture and Attention

The attention mechanism determines the importance of each component in a sequence relative to other components, allowing tokens to attend to other tokens and enabling different attention heads to focus on different aspects of the input.

Accessing LLMs Through APIs

Since running large language models locally is computationally intensive, the solution is to access them through APIs, where you send a message and receive a reply from cloud-hosted models.

Playground vs ChatGPT

OpenAI Playground is a developer-oriented testing environment with fine-grained control over model parameters and configurations, while ChatGPT is a consumer-facing chatbot with a simplified interface and pre-configured settings optimized for general conversations.

Alternative AI Playgrounds

Beyond OpenAI, there are several other platforms providing access to AI models, including AI Studio for Gemini, Claude API Console from Anthropic, Cohere Playground, and Hugging Face chat, each offering different models and capabilities.

API Integration with Python

The power of APIs lies in programmatic control over AI models, allowing developers to control thinking depth, model selection, token limits, output formats, and more through Python integration.

Response Parameters

Key parameters for controlling model responses include temperature (controlling randomness vs determinism), model selection, instructions (system prompts), and input messages, all of which can be fine-tuned for specific use cases.

GPT-5 New Features

GPT-5 introduced several new parameters including verbosity control (low/medium/high), freeform function calling for raw text payloads, Context-Free Grammar (CFG) for syntax constraints, and minimal reasoning mode for faster, deterministic tasks.

Message Roles in Conversations

The Chat Completions API uses three distinct roles:

- **developer** – highest priority instructions
- **user** – end-user instructions
- **assistant** – model-generated messages

This allows for structured multi-turn conversations.

Chat Completions vs Completions

Chat Completions maintain conversation state automatically, generating responses after reading the accumulated context, while Completions require you to provide the full conversation history each time.

Structured Output

To prevent hallucinations and ensure reliable data exchange, structured output using Pydantic classes ensures that AI responses match exact schemas, preventing application breaks from unexpected JSON keys or missing fields.

JSON Mode

JSON Mode is a more limited approach compared to structured output and should be avoided when possible, as it was an early solution that doesn't provide the same level of validation and reliability.

Day 5 | Speech Recognition Fundamentals

Introduction to STT & TTS

Speech-to-Text (STT) and Text-to-Speech (TTS) technologies enable AI models to understand and generate human speech, bridging the gap between natural language and digital systems.

Speech-to-Text (STT)

Basics of Digital Sound Digital sound is represented through sampling, where the sample rate (T) determines how many times per second the audio signal is measured. Higher sample rates capture more detail but require more storage and processing power.

Speech Recognition Challenges Speech recognition faces numerous challenges including the "cocktail party problem" with multiple speakers, code-switching between languages, domain-specific terminology, children's speech with different acoustic properties, emotional speech variations, and non-standard speech patterns like stutters or disabilities.

Transformers for Audio Audio transformers use two main architectures:

- **CTC (Connectionist Temporal Classification)** – encoder-only models like Wav2Vec2, HuBERT, and M-CTC-T that predict character-level outputs
- **Seq2Seq** – more flexible sequence-to-sequence mapping

CTC models are particularly useful for hands-on applications due to their direct character prediction approach.

State-of-the-Art Models

Model	Provider	WER	Notes
Whisper	OpenAI	4–8%	Trained on 680k hours
Conformer	Google	3–6%	Real-time capable
Wave2Vec 2	Meta	5–8%	Self-supervised
SpeechT5	Microsoft	4–7%	Unified model

Word Error Rate (WER) and Character Error Rate (CER) Word Error Rate measures transcription accuracy by comparing substitutions (S), deletions (D), and insertions (I) against correct words (C):

$$\text{WER} = (S + D + I) / (S + D + C)$$

Character Error Rate provides a more granular metric at the character level. WER is typically higher than CER since word-level mistakes compound character errors.

Word Accuracy Word Accuracy inverts the error rate: $\text{WAcc} = 1 - \text{WER}$. Higher values indicate better performance.

Whisper API Models

Model	Parameters	Speed
Tiny	39M	~32x

Base	74M	~16x
Small	244M	~6x
Medium	769M	~2x
Large	1550M	1x baseline

The choice between models balances processing speed against transcription accuracy.

Text-to-Speech (TTS)

TTS Use Cases and Importance TTS serves critical functions including accessibility for 285 million visually impaired people worldwide, educational support for dyslexia and language learning, content creation in the \$5 billion audiobook market, virtual assistants like Alexa and Siri, 24/7 customer service automation, and dynamic character voices in gaming and entertainment.

Issues with Natural Speech Natural speech generation faces complex challenges including emphasis variations (where the same sentence changes meaning based on stress), homophones requiring context to disambiguate (like "read" pronounced as "reed" vs "red"), prosody involving rhythm and intonation, emotional expression, and natural breathing patterns with pauses.

TTS Evolution Timeline

Era	Technology	Quality
1960s	Formant synthesis (DECtalk)	Robotic
1980s	Concatenative (AT&T Natural Voices)	Choppy
1990s	Diphone synthesis (Festival)	Improved
2000s	Unit selection (Apple's original Siri)	Natural-ish
2010s	Parametric (Google WaveNet)	High quality
2020s	Neural models (ElevenLabs, OpenAI)	Human-like

TTS Provider Comparison

Provider	Quality	Voices	Price/1M chars
OpenAI	Excellent	6	\$15–30
ElevenLabs	Best, voice cloning	100+	\$30–100
Google	Very good	400+ languages	\$4–16
Amazon Polly	Good	60+	\$4–24
Azure	Enterprise	400+	\$4–16
PlayHT	Ultra-realistic	800+	\$20–80

Ethics in TTS Ethical TTS practices require:

- Always obtaining consent for voice cloning
- Watermarking AI-generated content
- Disclosing AI voice usage
- Respecting voice actor rights
- Preventing misuse and deepfakes
- Protecting user privacy
- Monitoring for abuse